Clase 16 — Multiplexación, demultiplexación y UDP

Bloque: C (Transporte) · **Semana:** 6 · **Duración estimada:** 110 min **Correlativa:** Clase 15 (DHCP, configuración y simulación final) · **Sigue con:** Clase 17 (TCP: la cabecera campo por campo) **Material de referencia:** Tanenbaum cap. 6 (§6.1, §6.5) · RFC 768, 9293, 9000, 9868

🎯 Al terminar esta clase vas a poder

1. Explicar por qué hace falta el **puerto** como identificador de proceso en un host.
2. Distinguir **multiplexación** de **demultiplexación** y explicar la tabla de sockets.
3. Responder las **tres preguntas del examen 2010-12 P2** y explicar la diferencia con TCP.
4. Escribir la **cabecera de UDP campo por campo** y calcular un checksum de Internet a mano.
5. Justificar por qué "un socket UDP es una 2-tupla" es correcto pero incompleto.
6. Escribir en **C** un servidor UDP con `socket()`, `bind()`, `recvfrom()` y `sendto()`.

> **Prerequisito del examen:** `2010-12 P2` (8 pts, el problema central) · `2009-07 P2b` (8 pts, cuándo usar UDP) · `2010-12 Problema 1` (30 pts, sockets UDP en C).

PARTE 1 — TEORÍA

1.1 El problema: cuatro programas, una sola dirección IP

Firefox, correo, Spotify y un juego salen a la red **desde la misma máquina**: para el mundo hay una sola IP, `203.0.113.9`, y una sola MAC. Y sin embargo cada datagrama tiene que terminar en el proceso correcto. IP ya hizo su trabajo: llevó el paquete a la máquina correcta. Falta responder **¿a cuál de estos siete procesos?** (Firefox, correo, Spotify, juego, `sshd`, `dnsmasq`, `cupsd`). La respuesta clásica es el **número de puerto** (*port number*): un entero de **16 bits** (0 a 65 535) que la capa de transporte escribe en la cabecera y que el SO usa para elegir el proceso destino.

Tres propiedades del puerto que hay que tener presentes:

1. **El puerto es local a cada host.** El `53` del servidor DNS y el `53` del cliente no significan nada entre sí: un puerto es "un índice en la tabla de sockets de *mi* máquina".
2. **Identifica un servicio, no una persona.** No es seguridad: si tiene permisos, cualquier proceso puede hacer `bind()` sobre un puerto dado.
3. **El rango se reparte por registro:** bien conocidos (0–1023), registrados (1024–49151) y efímeros (49152–65535), que son los que el SO elige para los clientes.

Puerto y protocolo van juntos: **`53/udp` y `53/tcp` son dos servicios distintos**, y `443/tcp` es HTTPS mientras que `443/udp` es HTTP/3 (QUIC). Para tener a mano: 20/21 FTP, 22 SSH, 25 SMTP, **53 DNS (udp y tcp)**, 67/68 DHCP, 80 HTTP, 161/162 SNMP, 179 BGP, 853 DoT.

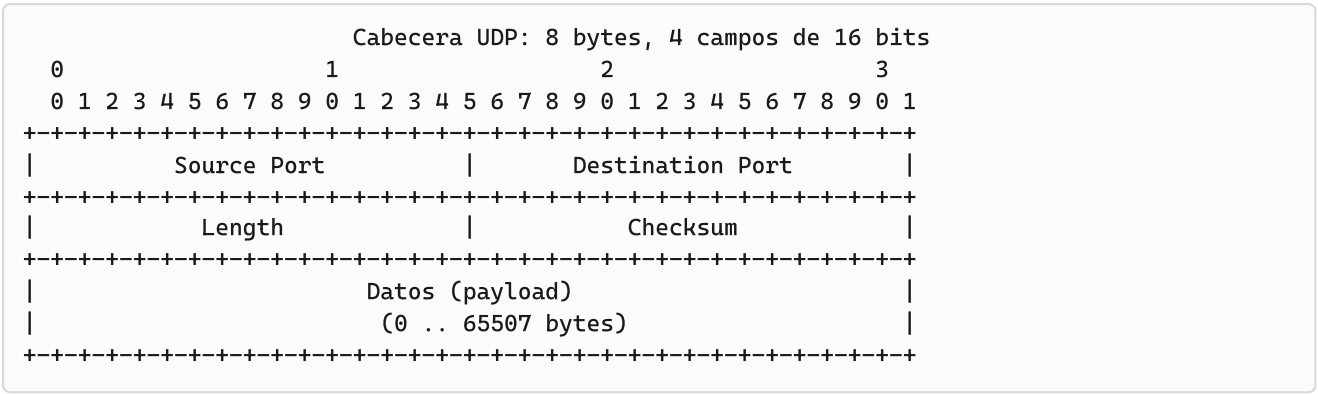
1.4 Por qué UDP no tiene conexión: lo que no hay

UDP es **sin conexión** (*connectionless*), **no confiable** (*unreliable*), también **best effort** ("lo mejor que se pueda"). La definición es negativa, y esa es la mejor forma de recordarlo:

```
┌──────────────────────── UDP: lo que NO hay ─────────────────────────┐
│ 1. NO hay handshake: ni SYN ni SYN-ACK. Sin estado ni seq inicial   │
│ 2. NO hay ACK ni retransmisión: si se pierde, se perdió.            │
│ 3. NO hay control de congestión ni de flujo: sin ventana ni MSS.    │
│ 4. NO hay orden: pueden llegar en el orden inverso.                 │
└─────────────────────────────────────────────────────────────────────┘
```

Y lo que sí tiene: **números de puerto**, **longitud** y un **checksum opcional**. La cabecera de UDP son **8 bytes**. Con 1000 B de payload, el datagrama completo pesa 1046 B con UDP (4,4 % de overhead) y 1058 con TCP (5,8 %): el 60 % del overhead de TCP son 12 bytes. Nadie elige TCP o UDP por el tamaño de la cabecera, sino por **las garantías que necesitás y el retardo que tolerás**.

1.5 La cabecera de UDP, campo por campo



Campo	Bits	Qué es
Source Port	16	Puerto del proceso que emite (12 345 = `0x3039`)
Destination Port	16	Puerto del proceso que recibe (53 = `0x0035`)
Length	16	Longitud de **cabecera + datos**, en bytes. Mínimo 8 (21 = `0x0015`)
Checksum	16	Detección de errores; **opcional en IPv4, obligatorio en IPv6**

El pseudo-cabecera

El checksum de UDP tiene que proteger el datagrama **y también las direcciones IP**: si un router se equivoca de dirección, el paquete llega al host equivocado y el checksum "daría bien" si solo mirara lo que viene después de IP. Pero **UDP no ve la cabecera IP** (se la pasa como un bloque opaco), así que el emisor **agrega las direcciones otra vez**, sintéticamente, al principio de lo que suma. A ese bloque se lo llama **pseudo-cabecera** (*pseudo-header*) y **no se transmite**: existe solo durante el cálculo.

IPv4 (12 bytes): `src` (4 B), `dst` (4 B), **1 byte de cero**, **protocolo = 17** y **longitud del datagrama UDP** (2 B). **IPv6 (40 bytes):** origen (16) + destino (16) + **longitud de capa superior de 32 bits** (4) + **3 bytes de cero** + **Next Header = 17**; con un túnel IPIP el Next Header es el de ese protocolo, y la longitud sigue siendo la del **datagrama UDP**.

El Internet checksum y el byte de relleno

Es el **Internet checksum** (RFC 1071), de **complemento a uno de 16 bits**: se toman los datos de a 2 bytes en *big-endian*, se suman, se devuelve cada acarreo al bit 0 hasta que no sobre (`suma = (suma & 0xFFFF) + (suma >> 16)`, repetir) y se toma el complemento del resultado; si da `0` se transmite `0xFFFF`, porque `0` significa "no computado" en IPv4.

El detalle que casi todos olvidan: si el total de bytes a sumar es **impar**, se agrega un byte `0x00` al final (siempre con UDP, porque la longitud puede ser impar). Calculado en el ejercicio 2.1. **Errores típicos:** olvidar ese byte; creer que avisa que el paquete no llegó (se **descarta en silencio**); y confundir el pseudo-cabecera con algo que viaja.

1.6 La 2-tupla y la 5-tupla: el matiz que hay que aclarar

```
Solución oficial del 2010-12:
  "Un socket UDP queda identificado completamente utilizando la IP del host
   y el número de puerto."                                  →  2-tupla
Lo que realmente usa el kernel:
  (protocolo, IP origen, puerto origen, IP destino, puerto destino)
                                                           →  5-tupla
```

Las dos cosas son ciertas y no se contradicen. La 5-tupla es la clave completa de un **flujo** en la red; la 2-tupla es la clave de **un socket local**.

- Un **socket UDP no conectado** (el caso normal de un servidor) está enlazado con `bind()` a un par `(IP local, puerto local)` y **solo uno** puede estar enlazado a ese par. Como todo lo que llega a ese socket es por definición "para" él, el kernel puede indexar la tabla **solo por (IP, puerto)** y aun así desambiguar: la 2-tupla **alcanza** porque los otros tres valores son constantes.
- En realidad la tabla se indexa por las cinco, y el `protocolo` está porque `53/udp` y `53/tcp` conviven sin cruzarse.
- Un **socket UDP conectado** (`connect()` sobre UDP existe y es real) **sí** se desambigua por la 5-tupla: el kernel **filtra** lo que entra y avisa con `ECONNREFUSED`.

> **La solución oficial ya lo anticipa en la parte b:** para elegir el socket TCP "en realidad alcanzaría con utilizar IP origen, Puerto origen, Puerto destino". Es el mismo razonamiento: **la clave de la red es la 5-tupla, pero como la IP destino y el protocolo están fijos en el host, se pueden omitir**. Si citás la 5-tupla, citá también por qué se puede reducir.

1.7 Comparación con TCP: el mecanismo equivalente

TCP demultiplexa **de otra manera**: usa un socket *escuchando* (*listening*) como punto de entrada y crea sockets nuevos por cada conexión.

```
navegador 1 ── socket <198.51.100.7,49200> ─┐
navegador 2 ── socket <198.51.100.7,49201> ─┴── socket ESCUCHANDO <0.0.0.0,80>
                                                  ▼   (el SO los crea)
                                            proceso servidor
```

En tres pasos: (1) el proceso servidor hace `bind()` + `listen()` y queda **un solo socket escuchando**, con **una** fila en la tabla; (2) llega un SYN a ese puerto, el kernel matchea el socket escuchando, **no le entrega el SYN al proceso**, y al completarse el handshake de 3 vías crea **un socket nuevo** con la **4-tupla** `<198.51.100.7, 49200, 203.0.113.9, 80>`; (3) cada segmento con esos cuatro valores matchea esa fila y va al proceso con `accept()`. **La diferencia de fondo:** en UDP la demultiplexación es *a un proceso*; en TCP es *a una conexión*, y por eso TCP necesita la 4-tupla y el handshake.

1.8 ⚠️ Actualización a 2026

Sigue igual — de memoria: cabecera de **8 bytes y 4 campos de 16 bits** con la longitud incluida; UDP **sin conexión, sin ACK, sin retransmisión, sin control de congestión ni de flujo, sin orden**; checksum **con pseudo-cabecera**, **opcional en IPv4 y obligatorio en IPv6**; y **TCP = 4-tupla, socket UDP = 2-tupla**, que es lo que se sigue enseñando y preguntando.

Cambió o hay que corregirlo:

- `RFC 9868` **— *Transport Options for UDP*** (oct-2025, *Standards Track*) **actualiza la** `RFC 768`: define la ubicación, sintaxis y semántica de **opciones de transporte de UDP**, que van en el espacio sobrante (*surplus area*) **después de los datos del usuario y antes del final del datagrama IP** (hoy ese espacio solo lo ocupa el *padding* de Ethernet). *Consecuencia:* "UDP no tiene opciones" ya no es estructuralmente cierto, aunque en la práctica el cable siga viendo 8 bytes de cabecera. La especificación original es la `RFC 768`, de J. Postel, 1980, catalogada como **STD 6 / Internet Standard**.
- **La "2-tupla de UDP" es una simplificación que hay que matizar.** `ACTUALIZACIONES.md` §14 lo dice textualmente: la clave de demux real en el kernel es la **5-tupla**. Presentá las dos y explicá la reconciliación de §1.6.
- **QUIC rompe el modelo de la 4-tupla.** `RFC 9000` §5.1: la identidad de una conexión QUIC es el **Connection ID** (*CID*), de 0 a 20 bytes, que viaja en la cabecera de cada paquete. Da tres cosas: **muchas conexiones multiplexadas en el mismo 5-tupla** (ver `RFC 8085`), **migración de conexión** (cambia la IP o el NAT cambia el puerto y la conexión sobrevive) y **CID de largo 0** si solo se usa la 5-tupla, que entonces no habilita migración.
- **HTTP/3 es UDP de verdad** (`RFC 9114`, con `Alt-Svc` `RFC 7838` y `SVCB`/`HTTPS` `RFC 9460`). Lo que **no** es cierto es que sea "TCP con otra cabecera": no hay conexión TCP, no hay handshake de 3 vías y no hay retransmisión de conexión; la pérdida se resuelve por *stream*. Cifras de 2026 (Cloudflare Radar): HTTP/2 50 %, HTTP/1.x 29 %, **HTTP/3 21 %**. Y sniffear HTTP/3 es difícil: va cifrado y sus cabeceras van en QPACK.
- **MPTCP (`RFC 8684`, 2020)** agrega multipath sin cambiar la demux de capa 4: cada *subflow* sigue con su 4-tupla. Desplegado por Apple y operadores, con **cuota marginal** (~0,4 % de bytes pico). Y ojo con **dual-stack**: `::ffff:a.b.c.d` y `IPV6_V6ONLY=0` afectan directo a la demultiplexación, porque `::ffff:203.0.113.9:53` y `203.0.113.9:53` deben llegar al mismo socket.

Consecuencia para el examen: la pregunta del 2010-12 P2 **sigue siendo la misma** y la solución oficial **sigue siendo válida**. Si sobra tiempo, una línea final: "*en el kernel la clave es la 5-tupla; la 2-tupla alcanza para un socket UDP no conectado porque el resto es constante, y QUIC ya multiplexa conexiones con Connection ID sobre el mismo 5-tupla.*"

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — El Internet checksum de UDP, calculado a mano

Datos. Datagrama IPv4 de `203.0.113.9` a `198.51.100.7`, protocolo 17. Puerto origen **12345**, puerto destino **53**, longitud **21**. Datos: la cadena ASCII `PRUEBA-EXAMEN`, de **13 bytes**.

a) Palabras de 16 bits a sumar. **b)** Suma mostrando cada acarreo. **c)** ¿Cuál es el checksum? **d)** Verificación en el receptor. **e)** Se alteran dos bits: el bit 0 de la palabra 1 y el de la palabra 3 de la pseudo-cabecera. ¿Lo detecta?

a) Las palabras a sumar, en orden:

```
Pseudo-cabecera (6 palabras)   1 CB00   2 7109   3 C633   4 6407   5 0011   6 0015
Cabecera UDP (3; el checksum va en 0)
                                7 3039   8 0035   9 0015
Datos (7: 6 parejas + 1 con el byte de relleno)
 10  5052
 11  5545
 12  4241
 13  2D45
 14  5841
 15  4D45
 16  4E00
```

La longitud total es 21 bytes, que es **impar**: por eso la última palabra es `4E00`. Ese byte de relleno **no viaja**: se inventa para poder sumar de a dos.

b) La suma. Si el resultado pasa de `FFFF`, se devuelve el acarreo y se sigue:

```
 CB00 + 7109 = 13C09  → acarreo 1 →  3C0A
 3C0A + C633 = 1023D  → acarreo 1 →  023E
 023E + 6407 = 06645  →              6645
 6645 + 0011 = 06656  →              6656
 6656 + 0015 = 0666B  →              666B
 666B + 3039 = 096A4  →              96A4
 96A4 + 0035 = 096D9  →              96D9
 96D9 + 0015 = 096EE  →              96EE
 96EE + 5052 = 0E740  →              E740
 E740 + 5545 = 13C85  → acarreo 1 →  3C86
 3C86 + 4241 = 07EC7  →              7EC7
 7EC7 + 2D45 = 0AC0C  →              AC0C
 AC0C + 5841 = 1044D  → acarreo 1 →  044E
 044E + 4D45 = 05193  →              5193
 5193 + 4E00 = 09F93  →              9F93
 ──────────────────────────────────────────
 suma final = 0x9F93
```

c) El checksum es el complemento a uno de 16 bits:

El checksum es el complemento a uno de 16 bits: `0xFFFF − 0x9F93 = 0x606C`. El datagrama queda `30 39 00 35 00 15 60 6C | 50 52 55 45 42 41 2D 45 58 41 4D 45 4E`.

d) Verificación en el receptor. El receptor suma **todo, incluido el campo checksum**: `9F93 + 606C = FFFF`, sin acarreo, y el complemento de `FFFF` es `0x0000`. **Un resultado `0x0000` significa "el datagrama está íntegro"**: ese es el criterio completo. Si el complemento no da cero, hay error y el datagrama se descarta en silencio.

Para el ítem (e): si el bit 0 de la palabra 1 pasa a 1 y el de la palabra 3 pasa a 0, los errores se compensan (`+1 − 1`), la suma vuelve a dar `0x9F93`, y el receptor calcula `0x606C`, que coincide con el checksum recibido: **no lo detecta**.

2.2 Ejercicio 2 — UDP sobre IPv6: pseudo-cabecera de 40 bytes

Enunciado. Un host `2001:db8::9` manda a `2001:db8::7` un datagrama UDP de **13 bytes** de datos ("PRUEBA-EXAMEN"), puerto 12345 → 53, IPv6 sin cabeceras de extensión. **a)** Escribí el pseudo-cabecera. **b)** Calculá el checksum. **c)** ¿Y si va dentro de un túnel IPIP? **d)** ¿Es opcional el checksum?

a) El pseudo-cabecera IPv6 (40 bytes = 20 palabras de 16 bits):

`0–15` Source `2001:0db8:...:0009` · `16–31` Destination `2001:0db8:...:0007` · `32–35` Upper-Layer Packet Length `0x00000015` = 21 · `36–38` Zero = `0x000000` · `39` Next Header = 17 (UDP).

b) Checksum. Con las mismas palabras del ejercicio 2.1 pero reemplazando las 6 del pseudo-cabecera IPv4 por las 20 del IPv6: `suma = 0x94D0` y `checksum = ~0x94D0 = 0x6B2F`. La aritmética es **exactamente la misma** que en IPv4; solo cambia el pseudo-cabecera. Fijate que la longitud que va en el pseudo-cabecera es **la del datagrama UDP (21)**, no la del paquete IPv6 (que sería 40 + 8 + 13 = 61). **Errores típicos:** poner la del paquete IPv6, u olvidar los 3 bytes de cero (el pseudo-cabecera IPv6 tiene 40 bytes exactos).

c) Con IPIP el Next Header **no es 17** sino el del protocolo de encapsulación (IPIP = 4), y la longitud sigue siendo la del **datagrama UDP interno**. **d)** En IPv6 el checksum es **obligatorio**: un `0` recibido implica descartar el datagrama, mientras que en IPv4 el `0` significa "no computado" y es legal (muchos stacks antiguos lo dejaban en cero para ahorrar CPU).

2.3 Ejercicio 3 — Sockets en C: un servidor UDP que recibe y responde

Enunciado. Escribí un servidor UDP en C que atienda el puerto 12345, responda `OFERTA <precio>` a los pedidos `OFERTA` y `COMMIT` o `REFUSE` a los `CONNECT` si se alcanzó el máximo de usuarios.

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>      /* + sys/types.h, netinet/in.h */
#include <netinet/in.h>

#define PUERTO 12345
#define MAX_MSG 128
#define MAX_USERS 20
#define PRECIO 150

static int usuarios = 0;

int main(void)
{
    int s;
    struct sockaddr_in local, cliente;
    socklen_t len_cli;
    char buf[MAX_MSG];
    ssize_t n;

    s = socket(AF_INET, SOCK_DGRAM, 0);          /* SOCK_DGRAM = UDP */
    if (s < 0) { perror("socket"); exit(1); }

    memset(&local, 0, sizeof local);
    local.sin_family      = AF_INET;
    local.sin_port        = htons(PUERTO);         /* orden de red: big endian */
    local.sin_addr.s_addr = htonl(INADDR_ANY);    /* todas las interfaces */
    if (bind(s, (struct sockaddr *)&local, sizeof local) < 0) { perror("bind"); exit(1); }

    for (;;) {
        const char *resp;
        len_cli = sizeof cliente;                  /* recvfrom: datos + emisor */
        n = recvfrom(s, buf, sizeof buf - 1, 0,
                     (struct sockaddr *)&cliente, &len_cli);
        if (n < 0) { perror("recvfrom"); continue; }
        buf[n] = '\0';

        if (strcmp(buf, "OFERTA") == 0) {
            n = snprintf(buf, sizeof buf, "OFERTA %d", PRECIO);
            resp = buf;
        } else if (strcmp(buf, "CONNECT") == 0) {
            if (usuarios < MAX_USERS) { usuarios++; resp = "COMMIT"; }
            else                      { resp = "REFUSE"; }
        } else {
            resp = "ERROR";
        }
        /* se responde AL QUE MANDO: por eso guardamos `cliente` */
        if (sendto(s, resp, strlen(resp), 0,
                   (struct sockaddr *)&cliente, len_cli) < 0) {
            perror("sendto");
        }
    }
    return 0;
}
```

El cliente `connect()` **conecta** su socket y puede usar `send()` / `recv()`: eso fija el par remoto por defecto, hace que el kernel **descarte** lo que venga de otro sitio y devuelve un ICMP 3/3.

Las tres ideas del ejercicio, que son las que se preguntan:

1. `socket(AF_INET, SOCK_DGRAM, 0)` → **UDP**; con `SOCK_STREAM` sería TCP. Y `bind()` es casi obligatorio en un servidor, porque **`recvfrom()` solo funciona sobre un socket enlazado**.
2. `htons()` / `htonl()`: los enteros multi-byte viajan en **big endian** (*network byte order*) y la familia `h` (*host to network*) convierte.
3. **El kernel guarda la dirección de quien mandó**, y por eso la respuesta puede ir al puerto efímero que ese proceso chose: ese es el mecanismo de "responder sin conocer el puerto del otro".

Errores típicos (los del C de la solución oficial de 2010-12): comparar cadenas con `==` (compara **direcciones de memoria**, no contenido; usá `strcmp()`); usar una variable sin inicializar como argumento de entrada; y `strlen()` donde va el largo real, que devuelve `snprintf()`.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-12, Pregunta 2 (8 puntos) — Multiplexación/demultiplexación

> **Fuente:** Examen `eirc1012` / `sirc1012` — 17 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf

Enunciado (texto del examen)

> a) Describa el mecanismo de multiplexación/demultiplexación para sockets UDP, discutiendo los siguientes puntos: 1. Cuando un host recibe un segmento UDP, ¿de qué manera decide a qué proceso derivárselo? 2. Cuando el proceso recibe el segmento, ¿cómo sabe a dónde enviar la respuesta? 3. ¿Con qué información queda completamente identificado un socket UDP en la red?
>
> b) Indique la diferencia entre el mecanismo de la parte a) y el utilizado para sockets TCP. ¿Con qué información queda completamente identificado un socket TCP en la red?

Solución oficial (verbatim)

> **a)** Cuando un host recibe un segmento UDP, utiliza el puerto destino indicado en el segmento para determinar a qué proceso debe enviárselo. El proceso que lee el segmento utiliza los campos IP origen y puerto origen para saber cuál es el socket UDP donde debe enviar la respuesta. Un socket UDP queda identificado completamente utilizando la IP del host y el número de puerto.
>
> **b)** El caso de TCP es un poco diferente: un proceso estará esperando conexiones en un puerto; cuando se establece la conexión, el sistema operativo creará un socket que enlaza , y se lo devolverá al proceso para que lo atienda; cuando llega un segmento TCP al host, se utilizará los cuatro valores para determinar a cuál de los sockets hay que derivar el segmento (en realidad alcanzaría con utilizar IP origen, Puerto origen, Puerto destino). Un socket TCP queda identificado completamente utilizando los cuatro campos: IP de origen, Puerto de origen, IP de destino, Puerto de destino.

Análisis de la solución oficial

La respuesta oficial acierta el mecanismo de los tres puntos y el **b)** es impecable en lo esencial; lo mejor es el **a)2**, porque demuestra que el sistema es **simétrico**. Lo que **falta** es la frase que da el punto en **a)1** (*el SO mantiene una tabla de sockets y busca la entrada que coincida*), una mención al **pseudo-cabecera** (otro uso de la IP de origen en UDP) y el matiz de que la **2-tupla es una simplificación** (§1.6).

Mi resolución, punto por punto

a)1 — ¿Cómo decide a qué proceso derivar el segmento? El SO mantiene una **tabla de sockets**. Cuando llega un datagrama con protocolo UDP: (1) toma el **número de protocolo** (17) y busca, **en la tabla de sockets UDP**, la fila cuyo `(IP local, puerto local)` coincida con el `(IP destino, puerto destino)` del datagrama — TCP tiene su propia tabla, y por eso `53/udp` y `53/tcp` no se cruzan; (2) si hay coincidencia, **entrega los datos al proceso** con `recvfrom()`, que devuelve la IP y el puerto de origen; (3) si no hay coincidencia, **descarta** y devuelve **ICMP tipo 3, código 3**.

a)2 — ¿Cómo sabe a dónde enviar la respuesta? Por **simetría del protocolo**: cuando el SO entrega el datagrama al proceso, le pasa también los dos campos de la cabecera, la **IP de origen** y el **puerto de origen**. Con esos dos datos más su propia IP y puerto local (que ya conoce) tiene los cuatro valores que alcanzan:

```
Respuesta = a la IP origen del datagrama, con puerto = puerto origen del datagrama
```

a)3 — ¿Con qué información queda identificado un socket UDP? La oficial: **la IP del host y el número de puerto** (la 2-tupla). Es correcto para un socket **no conectado**, y hay que justificarlo: como está enlazado a un par `(IP, puerto)` y **solo puede haber uno por par**, los otros tres valores son constantes. La respuesta completa, la de 2026: la clave de un **flujo en la red** es la **5-tupla** `(protocolo, IP origen, puerto origen, IP destino, puerto destino)`, y recién con `connect()` el kernel pasa a filtrar por ella completa.

b) — Diferencia con TCP y su identificación. Tres diferencias: (1) **el punto de entrada es uno solo y lo atiende el SO**, que maneja los SYNs sin despertarse el proceso; (2) **cada conexión crea un socket nuevo** con la **4-tupla** `<IP origen, puerto origen, IP destino, puerto destino>`, que se le pasa al proceso con `accept()`, mientras el socket escuchando sigue ahí; (3) **la clave es más grande porque hay más que distinguir**: contra el mismo servidor 80, la IP destino y el puerto destino son **siempre los mismos**, así que lo único que distingue las conexiones es el **par de puertos**. Por eso la solución oficial notó que alcanzaría con tres valores: el protocolo y la IP destino quedan implícitos. La especificación vigente es **`RFC 9293`** (2022, *Internet Standard*), que sustituye a la `RFC 793`.

3.2 Examen 2009-07, Pregunta 2b (8 puntos) — Cuándo usar UDP en vez de TCP

> **Fuente:** Examen `eirc0907` / `sirc0907` — 29 de julio de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0907.pdf

Enunciado (texto del examen)

> b) Mencione dos casos (aplicaciones, servicios, etc.) en los que resulta más apropiado usar UDP que TCP. Justifique su respuesta.

Solución oficial (verbatim)

> Por ejemplo: streaming de video y telefonía sobre internet. Son casos en los que el control de congestión de TCP o el tiempo consumido por el establecimiento de conexión es contraproducente por tratarse de aplicaciones de tiempo real en las que más vale perder algunos paquetes que atrasarlos todos.

Mi resolución, ampliada

La solución oficial acierta los dos casos y da **dos justificaciones correctas** (en el primero, la **oportunidad**; en el segundo, el **costo del handshake**). Para 8 puntos hay que dar **dos casos con su justificación específica**.

Caso 1 — Telefonía por Internet (VoIP) y videoconferencia. La aplicación genera un sample cada 20 ms y tiene que reproducirlo en su momento. Si uno se pierde, con TCP el stack **retransmite** el sample y, peor, **entrega los siguientes en orden**: los samples 4, 5 y 6 esperan en el buffer hasta que llega el 3, que llega 200 ms tarde y **ya no sirve para nada** (se descarta igual, pero después de acumular **jitter**). Con UDP se pierde y se sigue. **En una frase:** en tiempo real un dato tardío es peor que uno ausente, y TCP garantiza orden y no pérdida, justo lo que no querás.

Caso 2 — Consulta de DNS (y cualquier protocolo de consulta/respuesta corto). La conexión TCP necesita **1 RTT entero de handshake** antes de que el cliente pueda mandar la consulta, y **3 paquetes** de ida y vuelta para una respuesta de, digamos, 100 bytes; con UDP son **2 paquetes**, y el riesgo de perder la consulta se resuelve con un **reintento temporizado**. **El costo del establecimiento no se amortiza si la transacción dura un solo mensaje.**

> **La contraargumentación clásica:** muchas redes **bloquean el tráfico UDP saliente**, y por eso los protocolos que quieren las ventajas de UDP tunelizan UDP dentro de TCP: **HTTPS** lo hace con TLS y **QUIC** (`RFC 9001`) usa **TLS 1.3 sobre UDP**. Es la respuesta moderna a "pero UDP no es confiable": *la confiabilidad la aporta la capa de aplicación, no el transporte*.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Multiplexar** = muchos procesos → un flujo; **demultiplexar** = un datagrama → su proceso.
2. El **puerto** (16 bits) identifica el proceso **dentro del host**, no entre hosts.
3. Es una **búsqueda en la tabla de sockets** por `(protocolo, IP destino, puerto destino)`; si no hay coincidencia, se descarta y se avisa con ICMP 3/3.
4. La **respuesta** va a la IP y al puerto de origen: eso lo da el `recvfrom()`.
5. **Socket UDP = 2-tupla**; la clave real de un flujo es la **5-tupla** `(proto, srcIP, srcPort, dstIP, dstPort)`.
6. **Socket TCP = 4-tupla**, porque en un servidor hay muchas conexiones al mismo `IP:puerto` y lo único que las distingue es el par de puertos.
7. UDP: **8 bytes** de cabecera y ni una garantía más. *Best effort*.
8. El **checksum** usa un **pseudo-cabecera** (IP + protocolo + longitud) y, si la longitud es impar, se rellena con un byte `0x00`; en **IPv6** es obligatorio y en IPv4 opcional.
9. **QUIC** (`RFC 9000` §5.1) usa el **Connection ID** en vez de la 4-tupla.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un host tiene sockets ligados a `203.0.113.9:53` y `203.0.113.9:631`. Llega un datagrama con `dport = 53`. ¿Qué hace el SO y por qué no hay ambigüedad?

<details>
<summary>Respuesta</summary>
</details>

2. ¿Por qué un socket UDP conectado se identifica con la 5-tupla y uno sin conectar con la 2-tupla?

<details>
<summary>Respuesta</summary>
</details>

3. Llega un datagrama con checksum `0xABCD` pero el receptor calcula `0xABCE`. ¿Qué hace el receptor y por qué no avisa al emisor?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Hace falta byte de relleno en un datagrama UDP de 6 bytes de datos (longitud total 14)?

<details>
<summary>Respuesta</summary>
</details>

5. ¿Por qué el checksum de UDP incluye las direcciones IP si la cabecera IP ya viaja en el paquete?

<details>
<summary>Respuesta</summary>
</details>

6. En el ejercicio 2.2, ¿qué cambia si el datagrama viaja sobre IPv6 en lugar de IPv4?

▶ Respuesta

7. ¿Por qué TCP necesita la 4-tupla y UDP no? En una frase.

▶ Respuesta

8. ¿Qué rompe QUIC del modelo de la 5-tupla y cómo lo resuelve?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc1012` **/** `sirc1012` — P2 (8 pts, mux/demux UDP vs TCP) · Problema 1 (30 pts, sockets UDP en C)
- **Examen** `eirc0907` **/** `sirc0907` — P2b (8 pts, cuándo usar UDP en vez de TCP)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 — §6.1 (*Multiplexing*), §6.5 (*UDP*)
- `RFC 768` — *User Datagram Protocol* (1980, **STD 6 / Internet Standard*; verificado)** · `RFC 9868` — ***Transport Options for UDP* (oct-2025)** · `RFC 9293` — ***TCP* (2022; sustituye a la 793)** · `RFC 1071` ** — Internet checksum
- **QUIC/HTTP-3:** `RFC 9000` (transporte, §5.1 Connection ID) · `RFC 9001` (TLS 1.3) · `RFC 9114` (HTTP/3) · `RFC 7838` (Alt-Svc) · `RFC 9460` (SVCB) · `RFC 8085` (CID de largo 0)
- **Clase siguiente:** 17 — TCP: la cabecera campo por campo